

```

--
=====
-- SISCURATELA PRO - FASE 1: MODELAGEM DE BANCO DE DADOS (POSTGRESQL)
-- Arquitetura Data-Driven e Security by Design
--
=====

-- Habilitar extensão para geração de UUIDs (Identificadores únicos e seguros)
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

--
=====
-- BLOCO 1: SEGURANÇA, ACESSO E PRIVACIDADE
--
=====

-- 1. Definição dos Perfis de Acesso
CREATE TYPE tipo_perfil AS ENUM ('CURADOR_ADMIN', 'ADVOGADO_AUDITOR');

-- 2. Tabela de Usuários (Com proteção contra força bruta e senhas em Hash)
CREATE TABLE usuarios (
    id_usuario UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nome VARCHAR(150) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    senha_hash VARCHAR(255) NOT NULL, -- NUNCA salvar em texto puro (usar
Bcrypt)
    perfil tipo_perfil NOT NULL,

    -- Campos de Segurança e Recuperação
    tentativas_falhas INT DEFAULT 0,
    bloqueado_ate TIMESTAMP,
    token_recuperacao VARCHAR(255),
    expiracao_token TIMESTAMP,

    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Logs de Auditoria de Acesso (Exigência de Compliance)
CREATE TABLE logs_acesso (
    id_log UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_usuario UUID REFERENCES usuarios(id_usuario) ON DELETE CASCADE,
    ip_origem VARCHAR(45),
    acao VARCHAR(50) NOT NULL, -- Ex: 'LOGIN_SUCESSO', 'LOGIN_FALHA',
'RECUPERACAO_SENHA'
    data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

--
=====
-- BLOCO 2: DOMÍNIO JURÍDICO E CADASTRO BASE
--

```

```

=====

-- 4. Tabela do Curatelado
CREATE TABLE curatelados (
    id_curatelado UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nome VARCHAR(150) NOT NULL,
    cpf VARCHAR(14) UNIQUE NOT NULL,
    numero_processo VARCHAR(50) NOT NULL,
    vara_origem VARCHAR(150) NOT NULL,
    data_inicio_curatela DATE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 5. Contas Bancárias e Centros de Custódia (Obrigatório conciliação MPDFT)
CREATE TABLE contas_bancarias (
    id_conta UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_curatelado UUID REFERENCES curatelados(id_curatelado) ON DELETE CASCADE,
    banco VARCHAR(100) NOT NULL,
    agencia VARCHAR(20) NOT NULL,
    numero_conta VARCHAR(50) NOT NULL,
    tipo_conta VARCHAR(50) NOT NULL, -- Ex: 'Corrente', 'Aplicação', 'Caixa
Físico'
    saldo_inicial NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    ativo BOOLEAN DEFAULT TRUE
);

--
=====
-- BLOCO 3: DICIONÁRIO DE DADOS (TAXONOMIA DO MPDFT)
--
=====

-- 6. Categorias Macro (Ex: Saúde, Moradia, Alimentação)
CREATE TABLE categorias_macro (
    id_macro UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nome VARCHAR(100) UNIQUE NOT NULL,
    tipo VARCHAR(20) CHECK (tipo IN ('RECEITA', 'DESPESA')) NOT NULL
);

-- 7. Categorias Micro (Ex: Farmácia, Plano de Saúde, Acompanhante, Internet)
CREATE TABLE categorias_micro (
    id_micro UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_macro UUID REFERENCES categorias_macro(id_macro) ON DELETE RESTRICT,
    nome VARCHAR(100) NOT NULL,
    UNIQUE (id_macro, nome) -- Impede categorias duplicadas dentro do mesmo
macro
);

--
=====
-- BLOCO 4: TRANSAÇÕES FINANCEIRAS E COMPROVANTES
--

```

```

=====

-- 8. Tabela de Transações (O Coração da Prestação de Contas)
CREATE TABLE transacoes (
    id_transacao UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_conta UUID REFERENCES contas_bancarias(id_conta) ON DELETE RESTRICT,
    id_usuario UUID REFERENCES usuarios(id_usuario) ON DELETE RESTRICT, -- Quem
registrou? (Allan ou Itamar)
    id_micro UUID REFERENCES categorias_micro(id_micro) ON DELETE RESTRICT,

    data_transacao DATE NOT NULL,
    valor NUMERIC(15, 2) NOT NULL CHECK (valor > 0), -- Impede valores zerados
ou negativos
    descricao TEXT NOT NULL,

    documento_referencia VARCHAR(100), -- Ex: Número da Nota Fiscal, Número do
Recibo
    exige_alvara BOOLEAN DEFAULT FALSE, -- Trava para operações sensíveis

    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 9. Repositório de Comprovantes (S3 / Google Cloud Storage)
CREATE TABLE comprovantes (
    id_comprovante UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    id_transacao UUID REFERENCES transacoes(id_transacao) ON DELETE CASCADE,
    id_usuario UUID REFERENCES usuarios(id_usuario) ON DELETE RESTRICT,

    nome_arquivo VARCHAR(255) NOT NULL,
    url_storage TEXT NOT NULL, -- Link seguro para o PDF (ex: AWS S3)
    tamanho_bytes INT,
    tipo_mime VARCHAR(50), -- Ex: 'application/pdf'

    ocr_processado BOOLEAN DEFAULT FALSE, -- Flag para saber se a IA já leu a
nota

    data_upload TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

--
=====
-- GATILHOS (TRIGGERS) PARA ATUALIZAÇÃO AUTOMÁTICA DE DATAS
--
=====
CREATE OR REPLACE FUNCTION atualiza_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.atualizado_em = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_atualiza_usuario

```

```
BEFORE UPDATE ON usuarios FOR EACH ROW EXECUTE PROCEDURE atualiza_timestamp();
```

```
CREATE TRIGGER trigger_atualiza_transacao
```

```
BEFORE UPDATE ON transacoes FOR EACH ROW EXECUTE PROCEDURE atualiza_timestamp();
```